AI-Enabled Developer

Module B – Context engineering

Level 1: Foundations · Topics B09–B13

Prompting is a subset. The real job is deciding what occupies the model's window at each step. Every slide in this module carries a worked example — the actual prompt, the actual output, and the actual numbers.

5	145	5	10
topics	minutes	concept diagrams	role profiles

© 2026 All rights reserved by Smart Working Solutions Ltd.

LEVEL 1: FOUNDATIONS

Module B at a glance

Five topics, five worked examples, roughly two and a half hours. Module A explained why context matters; Module B is how you control it.

B09 **Prompt structure** — Worked example · full width — 35 min

B10 **Output contracts and schemas** — Worked example · full width — 30 min

B11 **Few-shot examples at the edge** — Worked example · full width — 25 min

B12 **Write / Select / Compress / Isolate** — Worked example · full width — 35 min

B13 **Prompt anti-patterns** — Worked example · full width — 20 min

Role chips on every topic slide: FS Full-stack · BE Backend · FE Frontend · QA · DO DevOps/SRE · DE Data Eng · DS Data Sci/ML · PB Product/BA · SEC Security · EM Tech Lead

© 2026 All rights reserved by Smart Working Solutions Ltd.

Prompt structure

OUTCOME Write prompts with role, task, constraints, examples and stop conditions.

- Six slots: role, task, **constraints**, examples, **output contract**, stop conditions. The two in bold do most of the work.
- Constraints and explicit non-goals beat tone and persona. "No recommendations" changes the output more than any amount of role-play.
- Prompt-poking without hypotheses is gambling. Name what you expect to change **before** you change it.
- Four iterations should be enough. If it is not, the problem is context or task decomposition — not wording.

THE SAME TASK, TWO PROMPTS — REAL OUTPUT BOTH TIMES

BARE

```
Summarise this customer feedback.
```

What came back:

```
The feedback is generally mixed. Customers
appreciate the product quality but several
mention delivery issues. Overall sentiment
appears moderately positive, though there are
areas for improvement. I would recommend…
```

One paragraph · unbounded · recommendations nobody asked for

STRUCTURED — constraints + output contract

```
Summarise this feedback for a weekly ops review.
- Max 3 bullets, one line each
- No recommendations
- Flag any mention of a refund with [REFUND]
Return only the bullets.
```

What came back:

```
• Product quality praised in 7 of 12 responses
• Delivery delays in 5 of 12 — negative driver [REFUND]
• Packaging mentioned twice, neutral
```

Bounded · scannable · the [REFUND] flag is machine-parseable

EXERCISE Fix a deliberately bad prompt in four iterations or fewer, logging a hypothesis before each edit.

ROLES: FS · BE · FE · QA · DO · DE · DS · PB · SEC · EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Output contracts and schemas

OUTCOME Replace “please return JSON” with a schema constraint plus validation.

- "Return the sentiment as JSON" is a **request**. A schema is a **contract**. Use the contract — then validate anyway.
- Three things come back with the prose version that your parser has to survive: a preamble, code fences, and a trailing note.
- Retry-on-invalid: feed the validation error back and retry **once**, then fail loudly. Roughly fifteen lines of code.
- Log the invalid rate. It is your first free quality signal, and an early warning when something upstream drifts.

ASKING VS CONSTRAINING — AND WHAT THE REPAIR LOOP ACTUALLY SEES

ASKING

```
Return the sentiment as JSON.
```

```
Sure! Here’s the JSON you asked for:
```json
{ "sentiment": "positive" }
```
Let me know if you need anything else.
```

CONSTRAINING — schema enforced by the API

```
{"required": ["sentiment","mentions_refund"],
 "properties": {
   "sentiment": {"enum": ["pos","neg","mixed"]},
   "mentions_refund": {"type": "boolean"}}}
```

```
{"sentiment":"mixed","mentions_refund":true}
```

Parseable every time — and still validated in code

RETRY ON INVALID `ValidationError: 'sentiment' must be one of [pos,neg,mixed], got "positive"` → feed the error back → retry once → then fail loudly

EXERCISE Convert a prose-instructed prompt into a schema-constrained call; add a validator and observe the failure rate change.

ROLES: FS BE FE QA DO DE DS PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Few-shot examples at the edge

OUTCOME Select examples that teach the boundary rather than the average case.

- Two or three examples that show the **edge** beat ten that show the average.
- The three typical examples on the left are ones the model already gets right — you spent your budget confirming what it knew.
- The sarcastic review and the product-versus-service split are where the boundary actually is.
- Your examples double as documentation of your real decision rules — often the first time those have been written down.

THE SAME CLASSIFIER, TWO SETS OF EXAMPLES

THREE TYPICAL EXAMPLES

`"Great product, arrived fast."`	POSITIVE
`"Broken on arrival. Terrible."`	NEGATIVE
`"Does what it says. Fine."`	NEUTRAL

The model already gets all three right. Budget spent confirming what it knew.

TWO EXAMPLES AT THE EDGE

`"Well, the box arrived. Eventually. In three pieces. 10/10."`	**NEGATIVE** *sarcasm reads as praise*
`"Love the jacket. Delivery was a shambles."`	**MIXED** *product vs service split*

These two teach the boundary itself.

ACCURACY ON 20 HARD INPUTS typical examples 61% → **edge examples 89%** *measure on hard inputs only — the full set hides the effect*

EXERCISE Take a classifier prompt with three typical examples; replace them with two edge cases and measure the accuracy delta on hard inputs.

ROLES: **FS** **BE** FE **QA** DO **DE** **DS** PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Write / Select / Compress / Isolate

OUTCOME Name which of the four operations a context problem needs.

- **WRITE** — persist state outside the window. The style guide belongs in a rules file, not in every request.
- **SELECT** — retrieve narrowly. Three clauses, not the forty-page policy. Four fields, not the whole export.
- **COMPRESS** — summarise at boundaries, so last month's conversation does not ride along forever.
- **ISOLATE** — split across sub-agents with separate windows. These four survive every model rename.

A REAL PROMPT AUDIT — 8,320 TOKENS DOWN TO 2,660

WHAT WAS IN THE PROMPT		OPERATION AND FIX		AFTER
40-page returns policy, pasted whole	4,200	SELECT	retrieve the 3 relevant clauses	1,450
Entire 18-turn chat history	1,900	COMPRESS	rolling summary at each resolution	480
Full customer order export	900	SELECT	order_id, status, date, total	110
House style guide, inline	700	WRITE	move to a rules file, load on demand	0
Task, constraints and 2 examples	620		keep as is	620

8,320 tokens → 2,660 tokens 68% smaller, same accuracy. **Every cut names an operation — otherwise you are deleting and hoping.**

EXERCISE Take a bloated 8,000-token prompt and cut it by 60% without losing accuracy. State which operation you used at each step.

ROLES: FS BE FE QA DO DE DS PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

Prompt anti-patterns

OUTCOME Recognise and remove mega-prompts, “be accurate”, role-play cosplay and dump-everything.

- All four lines on the left are real shapes you will find in your own repo this week.
- "Be accurate and do not hallucinate" is a wish. Attaching the source and permitting refusal is a strategy.
- The mega-prompt buries instruction number seven. Six ordered constraints beat eight hundred words.
- And one that bites at Level 2: every tool definition you load is luggage the model carries before it reads the first user word.

FOUR REAL PROMPT LINES — AND WHAT TO WRITE INSTEAD

WRITTEN TODAY	WRITE THIS INSTEAD
× "You are a world-renowned, Nobel-prize-winning software engineer with 30 years of experience…" →	"Return 3 bullets. No recommendations. Flag anything mentioning a refund."
× "Be accurate and do not hallucinate." →	[attach the policy] "Answer only from the attached policy. If it is not there, say so."
× "…and also handle edge cases, and make it robust, and follow best practices, and…" (+840 more words) →	Six numbered constraints, ordered by importance. Nothing else.
× "Here is the full customer export for context:" { 4,200 tokens of CSV } →	"Customer 48213: status active, 3 open orders." (SELECT the four fields you need)

Token counts and accuracy figures on these slides are illustrative — run the comparison on your own prompts.

EXERCISE Audit three prompts from your own team and mark every anti-pattern you find.

ROLES: FS BE FE QA DO DE DS PB SEC EM

© 2026 All rights reserved by Smart Working Solutions Ltd.

PLANNING REFERENCE

Module B exercises

Exercises live here rather than on each topic slide. Each is repeated in the speaker notes of its topic.

ID	Topic	Exercise	Min
B09	**Prompt structure**	Fix a deliberately bad prompt in four iterations or fewer, logging a hypothesis before each edit.	35
B10	**Output contracts and schemas**	Convert a prose-instructed prompt into a schema-constrained call; add a validator and observe the failure rate change.	30
B11	**Few-shot examples at the edge**	Take a classifier prompt with three typical examples; replace them with two edge cases and measure the accuracy delta on hard inputs.	25
B12	**Write / Select / Compress / Isolate**	Take a bloated 8,000-token prompt and cut it by 60% without losing accuracy. State which operation you used at each step.	35
B13	**Prompt anti-patterns**	Audit three prompts from your own team and mark every anti-pattern you find.	20

© 2026 All rights reserved by Smart Working Solutions Ltd.

END OF MODULE B

What they can do now — and what comes next

AFTER MODULE B

- Write a prompt with role, task, constraints, examples, an output contract and stop conditions.
- Replace "please return JSON" with a schema constraint, a validator and a repair retry.
- Choose few-shot examples that teach the boundary rather than the average case.
- Name which of the four operations — write, select, compress, isolate — a context problem needs.
- Recognise and remove the four anti-patterns in prompts their own team uses today.

NEXT: MODULE C

AI-assisted development — the agent loop, repo instruction files, plan-then-execute, spec-driven development, agent-assisted TDD and choosing your tooling.

Module B taught you to control a single request. Module C is what changes when the model can run your tests and edit your files.

More context is not better context.

© 2026 All rights reserved by Smart Working Solutions Ltd.